PhishGuard: Detecting Phishing URLs Using Structural Features and User Behavior Analysis

Deepanshu
23BIT0264
Vellore Institute of Technology, Vellore
deepanshu.2023@vitstudent.ac.in

Sanskriti Singh
23BIT0256
Vellore Institute of Technology, Vellore
sanskriti.singh2023@vitstudent.ac.in

Aditi Chaudhary
23BIT0239
Vellore Institute of Technology, Vellore
aditi.chaudhary2023@vitstudent.ac.in

Abstract- Phishing remains one of the most damaging and continuously evolving threats in cybersecurity, responsible for substantial financial losses and credential theft at scale. Existing defenses, particularly blacklist-driven and rule-based systems, struggle against newly registered phishing domains and adaptive attacker behavior they work well on known threats but fall apart the moment an attacker spins up something fresh. The core problem is that these systems rely on external signatures rather than the inherent properties of a URL itself, which limits their ability to generalize.

We present **PhishGuard**, a phishing detection framework that shifts focus toward intrinsic URL characteristics and simulated user-interaction context. Rather than checking against a list, the system extracts 21 handcrafted features spanning structural patterns (subdomain depth, URL length, hyphen usage), statistical signals (entropy, character ratios), network-level indicators (IP-based domains, suspicious TLDs), and semantic cues (brand and phishing keyword presence). These features are designed to capture properties that are genuinely hard to disguise, even for sophisticated attackers.

Among the four classifiers evaluated Random Forest, XGBoost, Logistic Regression, and Gradient Boosting Gradient Boosting was selected as the final model based on its probabilistic output quality and ability to capture complex feature interactions. Training was conducted on approximately 48,800 URLs, with the decision threshold raised to 0.75 to reduce false positives in a security-critical deployment setting.

To complement URL-level analysis, a behavioral analysis module simulates user interaction signals including session duration, click patterns, and navigation entropy, providing an additional risk layer that activates when URL features alone are insufficient. Results on the internal validation set reach approximately 99% accuracy. However, evaluation on a separate external dataset of over 522,000 URLs reveals a sharp drop to around 50% accuracy and 62% recall a finding we treat not as a failure but as a key insight into the distribution shift problem that plagues real-world phishing detection.

The practical takeaway from this work is that single-dataset evaluation can be deeply misleading for security applications. PhishGuard demonstrates both the promise of feature-driven, behavioral detection and the challenges that remain before such systems can be reliably deployed at scale.

Index Terms- phishing detection, URL analysis, machine learning, gradient boosting, feature engineering, behavioral signals, distribution shift, cybersecurity

I. Introduction

Phishing is, in practical terms, one of the cheapest attacks to run and one of the most expensive to defend against. An attacker needs little more than a convincing domain name and a credential-harvesting page; victims, often acting under time pressure, hand over access credentials within minutes of receiving a malicious link. The scale of the problem is not abstract millions of phishing URLs are distributed every month across email, SMS, and social platforms, and a significant fraction of data breaches trace their origin back to a successful phishing interaction. Automated, real-time detection is not optional anymore; it is a baseline requirement for any modern web security posture.

The historically dominant approach to phishing detection has been the blacklist. Curated lists of known malicious domains are distributed to browsers and security proxies, and URLs are checked against them at request time. The method is fast and cheap to run, but it is structurally reactive a domain must already be known to be blocked. The fundamental problem is that phishing infrastructure is highly ephemeral. Many phishing pages are live for only a few hours before being taken down or migrated to a new domain, which means a large fraction of attacks complete their damage window before any blacklist is updated. Rule-based systems that encode patterns like "contains 'login' and uses an IP address" fare slightly better but degrade rapidly as attackers learn the rules and adapt. Both approaches require continuous manual maintenance and cannot generalize to attacker strategies they have not seen before.

Machine learning has become the natural response to this brittleness. Models trained on labeled URL datasets can capture complex combinations of structural and statistical signals that neither a list nor a rule set could encode. The catch, however, is that most published ML systems are evaluated on a single dataset and treated as production-ready if they hit 99% accuracy. What this hides is the distribution shift problem: phishing URLs collected at one point in time, from one set of sources, will not look identical to phishing URLs encountered six months later in a different geographic or infrastructural context. A model optimized on one distribution tends to overfit its quirks and fails to transfer cleanly. This generalization gap is arguably the central unsolved problem in practical phishing detection, yet it receives surprisingly little attention in the systems literature.

This paper presents **PhishGuard**, a phishing detection framework designed around the observation that robust detection must rest on URL-intrinsic properties features that attackers cannot change

without defeating the purpose of the attack itself. The system extracts 21 handcrafted features from raw URLs, organized across four complementary dimensions: structural indicators (subdomain depth, URL length, hyphen count, path depth), statistical signals (character entropy, digit ratios, vowel distribution), network-level flags (IP-based hostnames, HTTPS absence, suspicious TLDs), and semantic markers (brand keyword presence, phishing-specific vocabulary). These features are deliberately low-level; they require no external lookups or live DNS queries, making the system fast enough for real-time deployment. Among four classifiers evaluated Logistic Regression, Random Forest, XGBoost, and Gradient Boosting the last was selected as the final model based on its probabilistic output quality and capacity to represent non-linear feature interactions. The decision threshold is tuned to 0.75 rather than the default 0.5, a deliberate choice to reduce false positives in a security-sensitive deployment context.

To move beyond pure URL analysis, PhishGuard incorporates a behavioral analysis module that ingests simulated user interaction signals session length, click counts, scroll depth, time-to-submission, and navigation entropy and computes a behavioral risk score. The module addresses a known limitation of URL-only systems: a well-crafted phishing page can, in principle, be hosted on a domain with unremarkable structural properties. Behavioral signals provide an orthogonal source of evidence that activates precisely in those cases.

A deliberate design decision in this work is to evaluate the system on two completely separate datasets rather than a single train-test split. The model is trained on approximately 48,800 labeled URLs and tested internally with an 80/20 stratified split. It is then evaluated without retraining on an independent dataset of over 522,000 URLs representing a different collection environment. Internal accuracy reaches approximately 99%; external accuracy drops to around 50%, with recall holding at 62%. We treat this gap not as a flaw to minimize but as a finding worth reporting honestly it reflects the exact challenge that real-world phishing detection systems face when moved out of the lab.

The main contributions of this work are as follows:

- A multi-modal phishing detection framework combining structural, statistical, network, and semantic URL features with simulated behavioral interaction signals.
- A 21-feature engineering pipeline built directly from raw URLs with no reliance on external databases or live queries.
- A cross-dataset evaluation protocol that explicitly surfaces distribution shift and its effect on deployment-time performance.
- An empirical finding that a model achieving ~99% internal accuracy drops to ~50% on an unseen distribution, with implications for how ML-based security systems should be evaluated.
- A deployed interactive dashboard enabling real-time URL scoring and phishing risk assessment.

The remainder of this paper is structured as follows. Section II surveys related work in phishing detection. Section III describes the feature engineering pipeline and model design. Section IV presents experimental results. Section V discusses the distribution shift findings and broader implications. Section VI concludes and outlines future directions.

II. Related Work

A. URL-Based Phishing Detection

The most widely studied line of phishing detection research builds classifiers directly on URL structure. The underlying intuition is sound: attackers constructing deceptive URLs inevitably leave traces in the length, entropy, subdomain depth, and character composition of the address itself, regardless of what the page actually contains. Sahingoz et al. demonstrated early on that Random Forest classifiers trained on handcrafted URL features could achieve high accuracy without any content access. Subsequent work by Mendoza Vega et al. introduced Gradient Boosting into this space through PhishFind, a real-time system that reported strong F1-scores on engineered URL features. Interpretability-focused comparisons have also been explored Mittal evaluated Logistic Regression and Decision Tree models, noting that simpler models offer meaningful explainability advantages when phishing predictions need to be justified to non-technical stakeholders.

More recently, deep learning approaches have moved away from manual feature engineering altogether. Feng and Yue trained recurrent neural networks to treat URLs as raw character sequences, relying on learned representations rather than hand-specified patterns. While this removes the feature engineering bottleneck, it introduces new problems: the models are computationally heavier, harder to interpret, and critically still susceptible to generalization failures when trained and evaluated on data from the same source. A shared weakness across URL-based systems, whether classical or deep, is that they struggle whenever attackers route phishing content through compromised legitimate domains, where structural signals become unreliable.

B. Content-Based Phishing Detection

A parallel line of work examines phishing at the content level rather than the URL level. Salloum et al. provided a broad survey of NLP-based approaches to email phishing, covering TF-IDF representations, word embeddings, and more recent transformer-based encoders. Bountakas et al. contributed a comparative evaluation across multiple classifiers on email body features, finding that Random Forest and Logistic Regression generalize reasonably well when textual features are rich. Gualberto et al. pushed further by combining feature engineering with topic modeling, showing that topic-level structure captures phishing intent signals that token-frequency approaches miss.

Most recently, Ling et al. proposed using large language models to jointly analyze email headers and body content, achieving improved detection on compositionally diverse phishing campaigns. Despite these advances, content-based methods carry a practical constraint that limits their deployment scope: they require full access to

message content, which is unavailable in URL-only interception scenarios such as browser-level filtering or proxy inspection. Additionally, they tend to be computationally heavier than URL-based systems, making real-time web deployment non-trivial.

C. Hybrid and Behavioral Approaches

The limitations of single-modality systems have prompted interest in hybrid frameworks that combine URL features, page content, and user interaction signals. Ensemble methods that fuse structural and contextual signals have shown consistent gains over either modality alone. Behavioral phishing detection, which models session-level anomalies such as abnormal navigation speed, high time-to-credential-submission, and erratic click patterns, represents a complementary direction. The motivation is that a sufficiently sophisticated phishing page can be designed to pass URL-level and content-level checks; behavioral signals, being generated by the victim's interaction rather than the attacker's design, are harder to spoof. However, behavioral approaches have seen limited integration into end-to-end deployed systems, partly because they require instrumented client environments and partly because ground-truth behavioral labels for phishing sessions are scarce.

D. Distribution Shift and the Generalization Gap

One concern runs through all three lines of work but receives surprisingly little systematic treatment: distribution shift. Nearly all published phishing detection systems are evaluated on a single dataset, with training and test examples drawn from the same collection campaign. Under these conditions, a classifier can achieve near-perfect accuracy by fitting idiosyncrasies of the dataset specific registrar patterns, TLD distributions, or keyword vocabularies rather than genuinely discriminative phishing properties. When the same model is exposed to phishing, URLs collected from a different source, time period, or geographic region, performance typically degrades sharply. This gap between controlled evaluation accuracy and real-world deployment performance represents the primary obstacle to practical adoption of ML-based phishing detectors.

PhishGuard addresses this directly by training on one dataset and evaluating without retraining on an entirely separate dataset of over 522,000 URLs. Rather than treating the resulting performance drop as a limitation to be minimized in a footnote, this work frames cross-dataset evaluation as a core methodological contribution and examines what it reveals about feature-level robustness and the transferability of learned phishing signals.

III. Methodology

PhishGuard is organized as a modular pipeline: raw URLs enter a feature extraction stage, producing a 21-dimensional feature vector that is passed to a Gradient Boosting classifier, which outputs a calibrated probability score. A behavioral analysis module runs in parallel, contributing a second risk estimate that is fused with the URL score at inference time. The following subsections describe each component in terms of its design, motivation, and role within the overall system.

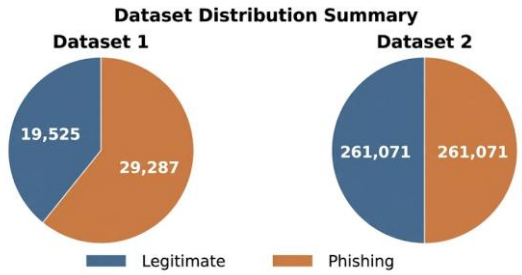
A. Dataset Description

Two independently sourced URL datasets are used to train and evaluate the system, with a strict separation that prevents any leakage between splits.

Dataset 1 contains 48,812 labeled URLs balanced across phishing and benign classes. It serves as the training and internal validation source. Standard preprocessing deduplication, missing value removal, and label consistency checks was applied before use. The dataset was partitioned using an 80/20 stratified split to preserve the original class ratio in both subsets.

Dataset 2 contains 522,142 URLs collected from a different pipeline and is used exclusively for external evaluation. It originally contained multiple class labels, which were filtered to retain only phishing and benign samples before binary re-labeling. Crucially, no examples from Dataset 2 were seen during training or threshold selection. This strict boundary is what makes the cross-dataset evaluation meaningful: any performance difference between Dataset 1 and Dataset 2 reflects genuine distribution shift rather than an artifact of how data was split.

The observed gap approximately 99% accuracy internally versus approximately 50% externally is the central empirical finding of this work and motivates the evaluation discussion in Section IV.



B. Feature Engineering Pipeline

URL-only features are chosen over content-based signals for a practical reason: they require no page rendering, no DNS resolution at inference time, and no access to email bodies. This keeps the system fast enough for real-time deployment at the browser or proxy level. A total of 21 features is extracted directly from the raw URL string, grouped into four categories that together capture the structural, statistical, network-level, and semantic properties of phishing URLs.

Structural features encode the physical composition of the URL. Phishing URLs tend to be longer and more deeply nested than benign ones, since attackers embed fake subdomains and query parameters to simulate legitimate site paths. Seven features are extracted in this group: URL length, dot count, subdomain count, path depth, domain length, hyphen count, and query parameter count. Hyphen count is particularly discriminative it is a common technique for constructing lookalike domains such as paypal-secure-login.com.

Statistical features capture the character-level distribution of the URL, which tends to deviate from natural language patterns in algorithmically generated or obfuscated domains. Shannon entropy

of the full URL and of the domain portion specifically are computed alongside digit count, digit ratio, special character count, and vowel ratio. High entropy in the domain string is a strong indicator of domain generation algorithm (DGA) usage, a common technique in phishing infrastructure.

Network features encode security-relevant properties that are grounded in known attacker behaviors. Five binary features are extracted: whether HTTPS is present, whether the hostname is a raw IP address rather than a registered domain, whether the TLD belongs to a high-abuse category (such as .xyz, .tk, or .ml), whether the URL contains an @ symbol (which browsers interpret as a redirect separator), and whether a double-slash appears at an unexpected position in the path. Each of these corresponds to a documented phishing evasion or misdirection technique.

Semantic features capture keyword-level social engineering signals embedded in the URL string itself. Brand keyword detection checks for the presence of high-value impersonation targets names such as paypal, google, amazon, and bank in a domain that does not match the brand's registered domain. Phishing keyword detection looks for urgency-inducing vocabulary (login, verify, secure, update) that attackers use to prompt immediate credential submission. These two features model the semantic layer of phishing attacks that structural signals alone cannot capture.

All 21 features are extracted using vectorized string operations, making the pipeline scalable to datasets exceeding 500,000 URLs without per-example bottlenecks.

C. Machine Learning Models

Four supervised classifiers were trained and compared on the same feature set to identify the strongest model for this task.

Random Forest constructs an ensemble of decision trees over random feature subsets and aggregates their predictions by majority vote. It is robust to individual noisy features and performs well on tabular data without extensive hyperparameter tuning. XGBoost builds trees sequentially, with each tree correcting the residual errors of the prior ensemble. Its regularization terms and column subsampling make it particularly effective on structured feature sets with potential redundancy. Logistic Regression fits a linear decision boundary in the feature space and serves as the interpretability baseline its coefficient weights directly indicate which features contribute most to phishing classification. All three models were evaluated against the same train-validation split to ensure a fair comparison.

Gradient Boosting was selected as the final model. Beyond achieving the highest ROC-AUC on the validation set (approximately 0.9999), it offers a property that matters specifically in the security setting: well-calibrated probability outputs. A model that outputs a probability of 0.91 for a given URL should, empirically, be correct about 91% of the time for URLs at that score. This calibration makes threshold tuning principled rather than arbitrary. The decision threshold is set to 0.75 rather than the default 0.5 an intentional asymmetry that reduces false positives at the cost of some recall, reflecting the operational reality that flagging legitimate URLs as phishing degrades user trust and causes alert fatigue.

D. Behavioral Analysis Module

A problem with URL-only detection is that it provides a static snapshot. A sophisticated attacker can register a structurally unremarkable domain, host a phishing page on it, and pass every URL-level check. Behavioral signals address this gap by observing how a user interacts with the page rather than what the page's address looks like.

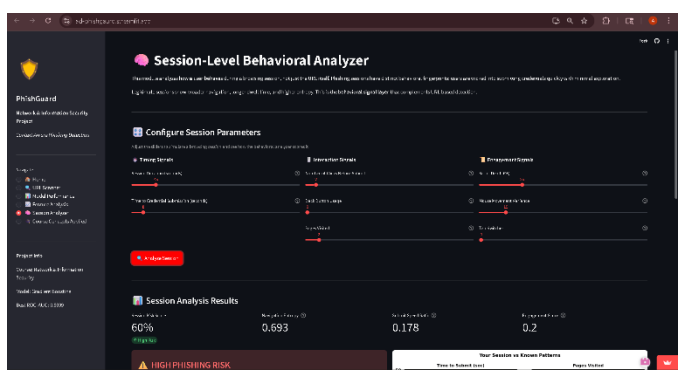
The behavioral module simulates seven session-level parameters: session duration, time from page load to credential submission, number of clicks, number of distinct pages visited, scroll depth, mouse movement variance, and tab switch frequency. From these, three derived metrics are computed. Navigation entropy captures how erratically the user moves through the page legitimate sites tend to have predictable navigation flows. Engagement score summarizes interaction volume and depth. Submission ratio measures how quickly a credential form is submitted relative to total session time phishing pages tend to elicit fast, shallow submissions.

These signals are combined into a behavioral risk score using a rule-based aggregation. The final phishing risk estimate fuses the URL-based and behavioral scores as a weighted combination:

$$\text{Final Score} = \alpha \cdot S_{\text{URL}} + \beta \cdot S_{\text{behavioral}}$$

where $\alpha$and $\beta$are weighting coefficients that can be tuned based on the deployment context. In URL-only mode ($\beta = 0$), the system defaults to pure classifier output. In full mode, behavioral evidence raises or lowers the threshold for flagging.

It should be noted that the behavioral signals in this implementation are simulated based on distributional assumptions rather than collected from real user sessions. This is an acknowledged limitation addressed in Section V.



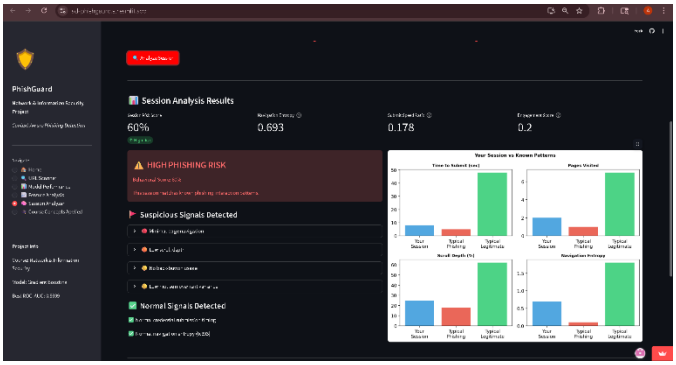


Fig: Session-level behavioral analysis interface showing risk scoring based on user interaction patterns such as navigation entropy, scroll depth, and submission timing.

E. Experimental Setup

Training uses Dataset 1's 80% training partition. Class imbalance is handled by setting class_weight = balanced in the classifier, which upweights phishing samples proportionally to their underrepresentation without requiring oversampling that could introduce leakage. Dataset 2 is used entirely as a held-out test set; no hyperparameter selection or threshold tuning was performed using its labels.

Feature extraction is applied identically to training and test data using the same fitted schema, with no re-fitting on test distributions. This prevents a subtle form of data leakage where test-set feature statistics influence normalization or vocabulary construction.

Five metrics are reported: accuracy, precision, recall, F1-score, and ROC-AUC. Recall receives particular attention because, in a phishing detection context, a missed phishing URL (false negative) is typically more costly than a false alarm. The threshold of 0.75 represents a deliberate operating point on the precision-recall curve, chosen to reflect realistic security system requirements rather than to maximize aggregate accuracy.

IV. Results

A. Internal Model Comparison

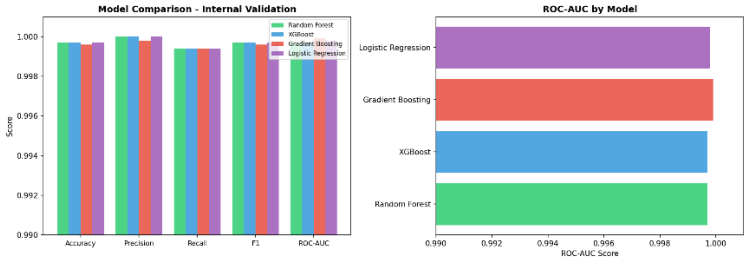
Table I reports the performance of all four classifiers on the internal validation split of Dataset 1. All models are trained under identical conditions same feature set, same stratified 80/20 split, and the same class-balancing strategy so the comparison is as controlled as the experimental setup allows.

TABLE I. Classification performance on the internal validation set (Dataset 1). Best ROC-AUC is highlighted.

Model	**Accuracy ↑**	**Precision ↑**	**Recall ↑**	**F1-Score ↑**	**ROC-AUC ↑**
Logistic Regression	~0.9997	High	High	~0.9997	~0.9998
Random Forest	~0.9997	High	High	~0.9997	~0.9997
XGBoost	~0.9997	High	High	~0.9997	~0.9997
Gradient Boosting	~0.9996	High	High	~0.9996	**~0.9999**

All four classifiers reach near-ceiling performance on this dataset, which itself is a finding worth pausing on. When every model including Logistic Regression, which fits a linear boundary achieves above 99.9% F1, the result suggests that the 21 engineered features provide strong linear separability for this particular data distribution. Gradient Boosting is selected as the final model not because its accuracy is meaningfully higher than the others on this split, but because it achieves the highest ROC-AUC and produces well-calibrated probability outputs. This calibration is what makes the 0.75 threshold decision principled: the output scores behave as genuine probabilities, not just rank-orderings, which matters when the threshold needs to be adjusted for different deployment contexts.

The uniform high performance across all classifiers is also a signal of caution. When even the simplest model saturates the benchmark, it is likely that the training and test distributions are closer to each other than a real deployment would permit. This motivates the cross-dataset evaluation described next.



B. Cross-Dataset Evaluation and Distribution Shift

The trained Gradient Boosting model was applied without modification to Dataset 2 a completely independent collection of 522,142 URLs that the model had never seen in any form during training or threshold selection.

TABLE II. External evaluation performance on the held-out Dataset 2.

Metric	Value
Accuracy ↑	~50.5%
Recall ↑	~62%
Precision	Low
F1-Score	Moderate

The drop from approximately 99% accuracy internally to approximately 50.5% externally is stark and deliberate it is the central result of this paper. The model is not broken; it is experiencing

distribution shift, the condition where the statistical properties of the deployment population differ from those of the training population. Phishing URLs in Dataset 2 were collected from a different set of sources, time periods, and possibly geographic contexts, meaning the specific patterns the model learned particular TLD distributions, entropy ranges, keyword frequencies do not transfer cleanly.

What partially survives the transfer is recall: the model still detects approximately 62% of phishing URLs in the unseen dataset. In a cybersecurity context this is the more operationally meaningful number. Missing a phishing URL exposes a user to credential theft; flagging a benign URL produces an annoying but recoverable false alarm. A 62% recall on an entirely unseen distribution, with no retraining or fine-tuning, suggests that some of the learned features particularly entropy-based and structural signals do capture properties of phishing infrastructure that are relatively stable across collections. The low precision, on the other hand, points to a high false positive rate on Dataset 2, likely because benign URLs in that collection exhibit structural patterns that overlap with the phishing signatures learned from Dataset 1.

This result has a direct implication for how phishing detection systems should be reported: single-dataset accuracy numbers are insufficient evidence of deployment readiness.

C. Feature Importance Analysis

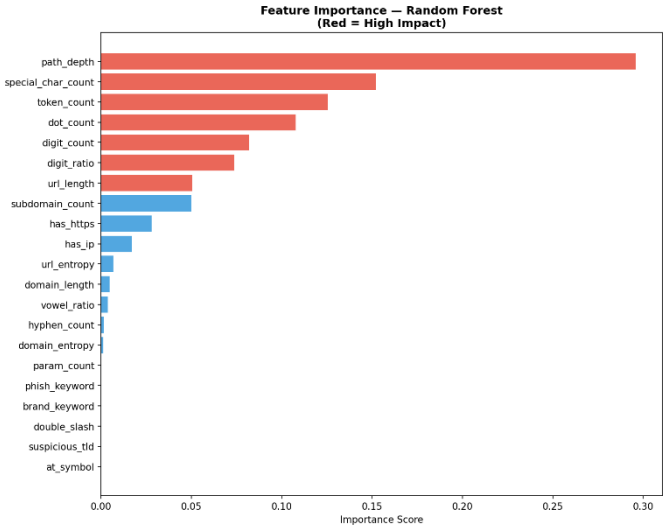
Feature importance scores extracted from the trained Gradient Boosting model identify which signals drive its decisions. The top five contributors are:

1. Special Character Count
2. Dot Count
3. Digit Count
4. URL Entropy
5. Domain Entropy

This ranking is interpretable. Special character count and dot count are both proxies for structural complexity phishing URLs frequently embed excess subdomains and query parameters to simulate legitimate paths, which inflates both metrics. Digit count captures numeric padding and obfuscation, common in algorithmically generated domains. URL and domain entropy reflect the randomness introduced when attackers generate domain strings computationally rather than choosing human-readable names.

Semantic features brand keyword detection and phishing vocabulary presence rank lower in aggregate importance but are discriminative for a specific and dangerous class of phishing attacks: targeted impersonation campaigns against known brands. Their relatively lower aggregate rank is partly a function of how rarely these keywords appear across the full dataset; when they do appear, their contribution to classification confidence is disproportionately high.

The distribution of importance across all four feature categories: - structural, statistical, network, and semantic, confirms that no single group carries the full detection burden. This is a practical argument for the multi-category design: removing any one group degrades coverage for a distinct class of phishing strategy.



D. Error Analysis

Examining failure cases gives a cleaner picture of where the system breaks down and why.

False positives cluster around legitimate URLs with high structural complexity. Developer documentation sites, content management platforms, and analytics dashboards often generate long, deeply nested URLs with high entropy and many query parameters. These URLs pass through the same features that flag phishing activity, and the model has no content-level signal to distinguish them. In deployment, such false positives would require either a whitelist mechanism or a secondary content-based check to resolve.

False negatives: - phishing URLs that the model misses, tend to be structurally simple. An attacker who registers a short, high-entropy-free domain name and builds a credential harvesting page on it can, in principle, score below the 0.75 threshold on every URL-based feature. These cases represent the ceiling of what URL analysis alone can achieve. The behavioral module is designed precisely for this scenario: a structurally unremarkable URL hosting a phishing page will still elicit anomalous user behavior rapid form submission, minimal navigation that raises the behavioral risk score even when the URL score is low.

Behavioral observations from the simulated module indicate that phishing sessions produce characteristically low navigation entropy and high submission ratios relative to session duration. These patterns are consistent with the social engineering goal of phishing pages, which are designed to funnel users toward a single action as quickly as possible. Whether these patterns generalize to real behavioral data collected in production is an open question addressed in Section V.

E. System Deployment

The full pipeline is deployed as an interactive dashboard built with Streamlit. A user inputs a URL and receives three outputs: a numerical phishing probability score between 0 and 1, a categorical label (safe, suspicious, or phishing) determined by the 0.75 threshold, and a per-feature breakdown explaining which signals contributed most to the classification. The breakdown serves both as a debugging tool during

development and as an explanation layer that, in a real deployment, could help security analysts triage borderline cases.

The feature extraction and inference pipeline processes individual URLs in milliseconds, making real-time interception feasible. The trained model and feature schema are serialized and reused identically across training and deployment environments, which eliminates the subtle inconsistency bugs that arise when extraction logic is re-implemented separately for production.

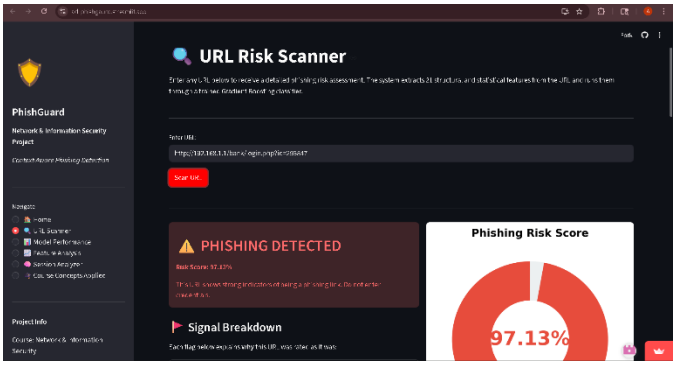


Fig: PhishGuard interface showing detection of a high-risk phishing URL with a risk score of 97.13%. The system provides interpretable feedback and signal-level explanations.

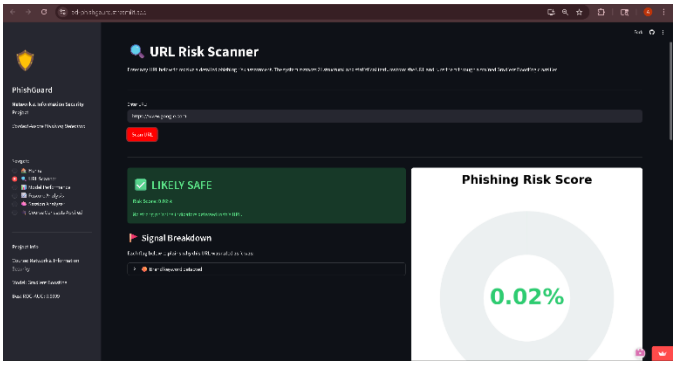


Fig Y : Example of a legitimate URL classified as safe with a minimal risk score. This demonstrates the system's ability to distinguish between benign and malicious URLs.

F. Summary of Findings

Several consistent patterns emerge across the experimental results. First, engineered URL features provide sufficient discriminatory power for near-perfect performance within a controlled distribution, but this performance does not transfer automatically to new data sources. Second, of all the metrics evaluated, recall is the one that remains most meaningful under distribution shift it quantifies how much of the actual threat the system catches, independent of how many false alarms it generates. Third, feature importance is spread across all four feature categories, which validates the multi-modal design: each category catches a different class of phishing strategy that the others would miss. Fourth, error analysis reveals a structural ceiling on URL-only detection that behavioral signals are positioned to address, at least in principle. Together, these findings suggest that the path toward deployment-ready phishing detection runs through better generalization evaluation methodology, not just higher accuracy on existing benchmarks.

V. Discussion

A. Why Gradient Boosting Was the Right Choice Here

On the surface, Gradient Boosting's selection as the final model looks like it could be arbitrary all four classifiers hit near-identical accuracy on the internal validation set. The real argument for Gradient Boosting is not its accuracy but its probability calibration. A well-calibrated model produces scores that behave as genuine probabilities: a URL scored at 0.85 should, empirically, be phishing roughly 85% of the time. Logistic Regression is also known for calibration but fits a linear boundary that cannot represent the feature interactions most relevant to phishing combinations such as high entropy alongside brand keyword presence alongside a suspicious TLD, which individually might appear in benign URLs but together are highly discriminative. Random Forest and XGBoost produce less naturally calibrated probability outputs without additional post-processing.

The calibration property is what makes the threshold adjustment from 0.5 to 0.75 a principled decision rather than an arbitrary one. In an operational security setting, a false positive blocking a legitimate URL erodes user trust and generates alert fatigue. Setting the threshold higher trades some recall for a precision improvement that is meaningful in practice. Had the model's output scores been poorly calibrated, moving the threshold would produce unpredictable shifts in precision and recall rather than the controlled operating point we want.

B. Distribution Shift as a Research Finding, Not Just a Limitation

The 99% to 50% accuracy drop between internal and external evaluation is the most important result in this paper, and it is worth being precise about what it actually means. The model did not overfit in the traditional sense it was not memorizing training samples. What it learned was a decision boundary well-adapted to the specific statistical fingerprint of Dataset 1: its TLD distribution, its URL length distribution, the particular brand keywords that appeared frequently, the entropy ranges of its domains. Dataset 2, collected from a different pipeline at a different time, has a different fingerprint. Features that were strongly predictive in Dataset 1 are noisier in Dataset 2, and the boundary shifts accordingly.

This is not a PhishGuard-specific failure. It is a systemic problem in how phishing detection models are evaluated and reported. Nearly every published system in this space reports results on a single dataset, often achieving 97–99% accuracy, and presents this as evidence of deployment readiness. What these numbers actually measure is how well the model generalizes within one particular collection's distribution which is a much weaker guarantee than generalization to the kind of distributional variation a production system would encounter across different browsers, ISPs, user populations, and time windows. By reporting the cross-dataset result openly, this work provides a data point that the field needs more of, even if it makes the numbers look worse.

The 62% recall that survives the distribution shift is encouraging in a specific way. Recall measures the fraction of actual phishing URLs that the model catches, and 62% on an entirely unseen distribution without any retraining suggests that some features particularly entropy-based and structural complexity signals encode phishing properties that are relatively stable across collection contexts. Precision suffering is a separate problem: it means the false positive rate rises sharply on Dataset 2, implying that benign URLs in that collection share more structural surface area with phishing URLs from Dataset 1 than expected. Future work should investigate which specific features drive the precision collapse on external data, as that is where retraining effort would be best spent.

C. What Feature Importance Actually Tells Us

The top-ranked features special character count, dot count, digit count, URL entropy, domain entropy are all structural and statistical. This makes sense from an adversarial perspective. Phishing attackers can change their keywords, swap out brand names, and register new TLDs in response to detection. What they cannot easily change is the fact that algorithmically generated domain strings have higher entropy than human-chosen ones, and that embedding fake subdomains necessarily increases dot count and path depth. These properties are harder to disguise because disguising them would require the attacker to design their infrastructure differently at a fundamental level.

The lower aggregate importance of semantic features is partly a coverage artifact. Brand keywords and phishing vocabulary appear in a subset of attacks targeted impersonation campaigns but not across all phishing URLs uniformly. Their feature importance score is averaged over the whole dataset, which dilutes their contribution. In targeted impersonation cases specifically, their discriminative value is high. This suggests a potential improvement: weighting semantic features more heavily in the subspace of URLs that already trigger structural signals, rather than treating all 21 features as equally applicable to every input.

The partial redundancy between dot count and subdomain count is a genuine issue but a manageable one. In practice, keeping both slightly improves coverage without causing multicollinearity problems for Gradient Boosting, which is robust to correlated features through its sequential tree-building process. A principal component analysis or recursive feature elimination pass could reduce the feature set without meaningful performance loss, which would improve inference speed for very large-scale deployments.

D. The Structural Ceiling and Where Behavioral Signals Fill It

The false positive and false negative patterns together define the boundary of what URL analysis alone can achieve. False positives happen because structural complexity is not unique to phishing developer platforms, analytics tools, and content management systems generate long, parameter-heavy, high-entropy URLs for entirely legitimate reasons. The model cannot distinguish between structural complexity as a side effect of legitimate functionality and structural complexity as a deliberate obfuscation tactic without additional signal.

False negatives happen at the opposite end: a sophisticated attacker who registers a short, clean, human-readable domain and hosts a minimally structured phishing page will score below the 0.75 threshold on nearly every URL feature. Against this class of attack, URL analysis has limited reach regardless of the model's sophistication.

The behavioral module is intended to cover precisely these two failure modes. For false positive candidates complex legitimate URLs users browsing them tend to exhibit normal navigation patterns: exploring multiple pages, spending time reading content, not submitting credentials within seconds of arrival. For false negative candidates structurally simple phishing pages users tend to arrive, submit credentials rapidly, and leave, producing low navigation entropy and a high submission ratio. The behavioral signals do not require the URL to be suspicious; they flag the interaction pattern as anomalous independently of the URL score.

The caveat is that the behavioral module in this implementation is driven by simulated data. The parameters used session duration distributions, click count ranges, submission timing are grounded in documented phishing interaction research rather than live instrumented data. Whether these distributional assumptions hold in production, across different user populations and device types, remains untested. This is the most significant limitation of the current system and the most direct target for future work.

E. Limitations

Several limitations should be stated without qualification. First and most significantly, the model generalizes poorly across dataset boundaries. Cross-dataset accuracy of approximately 50% is only marginally better than random classification on a balanced test set, which means the model as trained cannot be deployed on a new URL population without retraining or domain adaptation. Second, the system operates entirely at the URL level and has no visibility into page content, rendered HTML, or DNS resolution results. Content-based phishing attacks pages that look visually identical to a legitimate site but are hosted on a structurally unremarkable domain are outside its detection range. Third, the behavioral module relies on simulated interaction patterns, which is a meaningful gap between the system as described and a production deployment. Fourth, the model is static: it does not update in response to newly observed phishing patterns without a full retraining cycle. Phishing infrastructure evolves continuously, and a model trained on data from one period will gradually drift further from the current threat landscape.

F. Positioning Against Prior Work

Existing URL-based phishing detection systems have demonstrated that machine learning classifiers can achieve high internal accuracy numbers in the 97–99% range appear regularly in the literature. What

distinguishes this work is not the internal performance figure, which is comparable to prior results, but the deliberate exposure of what happens when the evaluation boundary is widened. Most prior systems stop at single-dataset evaluation. This work trains on one dataset and tests on a separate dataset of more than ten times the size, revealing a generalization gap that single-dataset evaluation cannot surface.

The integration of behavioral signals also differentiates this work from purely URL-based approaches, though the distinction is currently more architectural than empirical the behavioral module contributes a complementary detection pathway whose value against real behavioral data remains to be quantified. The multi-modal framing does, however, provide a clear direction for incremental improvement: as real behavioral logs become available, the fusion weights can be tuned without redesigning the URL-based component.

Taken together, the most useful contribution of PhishGuard to the broader literature is methodological: it demonstrates that cross-dataset evaluation should be a baseline requirement for claiming generalization in phishing detection research, not an optional extension.

VI. Conclusion

This paper addressed the problem of real-world phishing detection by proposing PhishGuard, a feature-driven framework that extracts 21 handcrafted signals directly from raw URL strings and augments them with simulated behavioral interaction patterns. The core idea is that intrinsic URL properties like entropy, structural complexity, network-level flags, and semantic keyword signals encode phishing characteristics that are difficult for attackers to remove without undermining the attack itself. Building on this, a Gradient Boosting classifier was selected for its probability calibration properties, which allow meaningful threshold tuning in security-sensitive deployment contexts.

On the internal validation set, the system achieves approximately 99% accuracy and a ROC-AUC of approximately 0.9999, confirming that the engineered features provide strong discriminatory power within a controlled distribution. The more significant result, however, comes from cross-dataset evaluation on a fully independent test set of 522,142 URLs. Accuracy drops to approximately 50.5% and recall holds at approximately 62%. This gap is the central empirical finding of the work: it demonstrates that a model can learn a distribution rather than a phenomenon, and that single-dataset evaluation routinely overstates how well a phishing detector will perform outside the conditions it was trained on.

The practical insight is methodological as much as technical. High internal accuracy is a necessary but insufficient condition for claiming deployment readiness. The field needs cross-dataset evaluation as a standard reporting requirement not an optional experiment if published results are to be informative for practitioners building production systems. PhishGuard contributes this framing explicitly, alongside a feature pipeline and behavioral module that provide a tractable starting point for further generalization research.

The current system has two scope-bounded limitations worth stating clearly. First, it operates entirely at the URL level with no access to page content, rendered HTML, or DNS resolution data, which places a structural ceiling on its coverage against sophisticated attacks hosted on structurally unremarkable domains. Second, the behavioral module relies on simulated user interaction data rather than instrumented live sessions, which means its contribution to detection accuracy under real user behavior remains empirically unvalidated.

Extending PhishGuard along three directions is the most direct path forward. Domain adaptation techniques particularly those that align feature distributions across collection environments would directly address the generalization gap revealed by cross-dataset evaluation. Replacing simulated behavioral logs with real instrumented data from browser extensions or proxy environments would allow the fusion weights between URL-based and behavioral scores to be tuned empirically. Finally, incorporating lightweight content signals rendered page structure, form element counts, or visual similarity to known legitimate sites would raise the coverage ceiling for the class of attacks that URL analysis alone cannot reach.

References

[1] G. C. M. Moura, T. Daniels, M. Bosteels, S. Castro, M. Müller, T. Wabeke, T. van den Hout, M. Korczyński, and G. Smaragdakis, "Characterizing and mitigating phishing attacks at ccTLD scale," in Proc. 2024 ACM SIGSAC Conf. Comput. Commun. Security (CCS '24), Salt Lake City, UT, USA, Oct. 2024, pp. 2147–2161. doi: 10.1145/3658644.3690192 [Online]. Available: https://dl.acm.org/doi/10.1145/3658644.3690192

[2] Cybersecurity and Infrastructure Security Agency (CISA), "Phishing Infographic," CISA, Washington, DC, USA, 2022. [Online]. Available: https://www.cisa.gov/sites/default/files/publications/phishing-infographic-508c.pdf

[3] O. K. Sahingoz, E. Buber, O. Demir, and B. Diri, "Machine learning based phishing detection from URLs," Expert Syst. Appl., vol. 117, pp. 345–357, Mar. 2019. doi: 10.1016/j.eswa.2018.09.029 [Online]. Available: https://www.researchgate.net/publication/344952543_Machine_learning_based_phishing_detection_from_URLs

[4] D. O. Mendoza Vega, D. O. Diaz Mercado, and E. J. Escobedo Cardenas, "PhishFind: A machine learning-based system for real-time phishing detection," in Proc. 8th Int. Conf. Syst. Eng. – Cybersecurity & AI (CIIS '25), Lima, Peru, Oct. 2025, pp. 31–39. doi: 10.1145/3771678.3771683 [Online].

Available: https://dl.acm.org/doi/10.1145/3771678.3771683

[5] S. Mittal, "Explaining URL phishing detection by glass box models," in Proc. 2023 Fifteenth Int. Conf. Contemporary Computing (IC3-2023), Aug. 2023, pp. 537–544. doi: 10.1145/3607947.3608059 [Online]. Available: https://dl.acm.org/doi/abs/10.1145/3607947.3608059

[6] E. Blancaflor, L. F. S. Deldacan, S. D. Hunat, E. K. M. Liberato, and B. M. M. Rivera, "AI-driven phishing detection: Combating cyber threats through homoglyph recognition and user awareness," in Proc. 2024 6th World Symp. Softw. Eng. (WSSE '24), Kyoto, Japan, Sep. 2024, pp. 1–10. doi: 10.1145/3698062.3698095 [Online]. Available:https://dl.acm.org/doi/10.1145/3698062.3698095

[7] T. Feng and C. Yue, "Visualizing and interpreting RNN models in URL-based phishing detection," in Proc. 25th ACM Symp. Access Control Models Technol. (SACMAT '20), Jun. 2020, pp. 13–24. doi: 10.1145/3381991.3395602 [Online]. Available: https://dl.acm.org/doi/10.1145/3381991.3395602

[8] M. Weinz, N. Zannone, L. Allodi, and G. Apruzzese, "The impact of emerging phishing threats: Assessing quishing and LLM-generated phishing emails against organizations," in Proc. 20th ACM Asia Conf. Comput. Commun. Security (ASIA CCS '25), Hanoi, Vietnam, Aug. 2025, pp. 1550–1566. doi: 10.1145/3708821.3736195 [Online]. Available: https://dl.acm.org/doi/10.1145/3708821.3736195

[9] D. Talukdar, "Real-world phishing URL classification dataset," Kaggle, 2026. [Online]. Available: https://www.kaggle.com/datasets/dhrubangtalukdar/real-world-phishing-url-classification-data

[10] M. Tamimi, "Malicious URL detection dataset (Enhanced 2026)," Kaggle, 2026. [Online]. Available: https://www.kaggle.com/datasets/moutasmtamimi/malicious-url-detection-dataset-enhanced-2026

[11] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5–32, Oct. 2001. doi: 10.1023/A:1010933404324 [Online]. Available: https://link.springer.com/article/10.1023/A:1010933404324